Complex & Intelligent Systems manuscript No.
(will be inserted by the editor)

Real-Time Intelligence and Scalable Architecture for Robotics

Anonymous Author(s)

the date of receipt and acceptance should be inserted later

Abstract Deploying learned perception in safety-critical robotic systems creates a fundamental tension: machine learning optimizes average-case accuracy, while physical systems demand worst-case timing guarantees. This paper addresses how these two worldviews can be reconciled through explicit, runtime-monitored contracts that couple adaptive inference quality to safety-critical execution envelopes.

This paper is a **survey/tutorial** with a **reference-specification** contribution: it treats real-time intelligence and scalable architecture as a single end-to-end systems problem spanning deadlines, execution-time variability, middleware quality of service (QoS), and the edge-cloud division of computation.

Two reference artifacts are specified: the **Temporal Intelligence Contract (TIC)** and **Contract-Aware Adaptive Inference (CAAI)**. TIC unifies latency envelopes, uncertainty/drift envelopes, safety fallback behavior, observability requirements, and scalability limits into a machine-readable contract intended for runtime monitoring and auditing. CAAI is a deterministic supervisor specification that consumes TIC-relevant signals (latency margin, calibrated uncertainty, and drift indicators) to switch between perception tiers and to couple tier selection to a corresponding safety envelope.

To support reproducible adoption, the paper provides a minimal contract schema, a conformance checklist, and reporting templates for prototype implementation and measurement. The goal is to make timing and learning-related assumptions explicit and auditable in learning-enabled robotic systems.

Keywords adaptive intelligent systems · uncertainty-aware control · emergent fleet behavior · runtime contract monitoring · real-time robotics · machine learning inference deployment · end-to-end latency monitoring · distributed robotic systems · edge robotics (edge computing) · digital twins for robotics ·

Affiliations withheld for double-blind peer review

scalability · ROS 2 (Robot Operating System 2) · robotics middleware · DDS/QoS

1 Introduction

Physical processes do not wait for computation. Cars keep moving, actuators keep applying force, environments keep changing, and the software must determine the next action through all of it. In any system coupled to the physical world, timeliness is part of correctness: a right choice that arrives after its deadline is, for the system, the same as a wrong one. A braking scenario makes this concrete. Under typical conditions, the perception-reaction-actuation chain must complete within a narrow window from the moment a driver spots a child stepping into the road. Stretch that chain by even a few seconds, and the outcome changes, the driver did not choose differently, the window for action had simply closed.

Real-time intelligence is the field that takes this constraint seriously, and this is why architecture is as important as algorithmic capability.

The increase in speed of control loops, combined with more complex decision-making and perception pipelines makes the problem more pronounced in robotics. A robot has to perform tasks with deadlines set by mechanics, human proximity, and task-specific contracts, but the software stacks that implement those tasks are now expected to scale from a single prototype to heterogeneous fleets. The scaling of team size, geography, and lifecycle implies new coordination and observability demands, which may subtly compromise timing guarantees unless they are explicitly built in.

Roadmap: This paper is a *survey/tutorial* with a *reference-specification* contribution: it synthesizes prior work, specifies contract artifacts and switching policies, and provides a prototype decomposition and pilot-experiment template (Section 6) intended to support reproducible evaluation in ROS 2-style systems. *Numeric examples in tables and worked calculations (except the discrete-event results in Section 6.6) are pedagogical illustrations of reporting format and contract semantics; deployed systems must be validated using the measurement protocol in Section 6.*

From a complex systems perspective, a robotic fleet under variable load exhibits emergent timing behavior: contention effects that are invisible in single-robot tests become dominant failure modes at fleet scale. TIC and CAAI are mechanisms for managing this emergent complexity—making adaptive, self-organizing behavior in the inference pipeline auditable and bounded rather than unpredictable.

It starts with a literature-based perspective of how real-time constraints and robot software architectures have developed together (Section 2), followed by making these constraints explicit through timing models, schedulability arguments, and worst-case reasoning (Section 3). Section 4 then treats scalability as a set of engineering decisions, examining how decomposition, middleware,

and the edge-cloud continuum either preserve or violate end-to-end latency guarantees.

Section 5 revisits learning from an operational perspective, focused less on model novelty and more on what it takes to run learned components inside a time-bounded pipeline, Section 7 covers safety, security, and energy as constraints that feed back into timing rather than as independent nonfunctional concerns. Section 8 presents deployed architectural patterns, Section 9 distinguishes this paper's contributions, and Sections 10 and 11 close with open problems and conclusions.

2 Literature Background and Conceptual Foundations

2.1 Survey methodology (scope and selection)

This article is written as a survey/tutorial with a reference-specification contribution. We focused on peer-reviewed literature and widely used engineering documentation across real-time systems, robotics middleware (ROS/ROS 2 and DDS), edge/cloud robotics, and learning-enabled autonomy. Sources were identified via keyword search and backward/forward citation chasing around core terms including *real-time robotics*, *WCET*, *tail latency*, *ROS 2 executor*, *DDS QoS*, *runtime verification*, *uncertainty calibration*, and *distributional drift*. We preferentially included papers that (i) report end-to-end latency behavior (including variance/tails), (ii) expose implementation-level factors (executors, scheduling, memory/IPC), or (iii) propose assurance mechanisms applicable to deployed systems (contracts, monitors, shields). Because the field evolves quickly, we also cite recent surveys and system reports where they provide an integrative view or standardized terminology.

2.2 Real-Time Computing and Control

The real-time systems community has developed a rich vocabulary for reasoning about when computation must finish, not just how fast it runs on average. This builds on classic work in schedulability, priority assignment, and worst-case execution time (WCET). Textbooks in that area distinguish hard deadlines, where a miss is catastrophic, from soft deadlines, where a miss only degrades quality [10, 25]. Robotics inherit this distinction in a straightforward way. Emergency braking is a hard deadline problem; missing it can cause direct harm. Keeping a semantic map fresh is generally soft, since a slightly stale map costs efficiency but rarely causes immediate danger.

A long-standing tension emerges when these timing concepts meet the assumptions of control. Classical control theory assumes periodic sampling because it makes stability analysis tractable. Real robot software does not behave that way: it is event-driven, irregular, and its workload shifts from one moment to the next, making timing analysis considerably harder [10, 27].

Event-based control paradigms (e.g., [4]) emerged in part to relax periodic-sampling assumptions. The gap has widened as learned components moved into perception and prediction pipelines. For instance, transformer-based perception models can have input-dependent attention costs and contend with control threads for shared GPU and memory bandwidth. Weakly hard constraints, which tolerate a bounded number of deadlines missed inside a sliding window, have re-emerged as a practical model for learned pipelines whose per-input runtime cannot be pinned down [6].

2.3 Middleware and Communication-Centric Design

Communication became a core architectural concern the moment robot systems grew beyond a single process. The Robot Operating System (ROS) made it standard practice to structure robot software as a graph of loosely coupled nodes exchanging messages, backed by a tooling ecosystem that made integration and reuse practical [34]. ROS 1 was not designed around deterministic timing, so latency variance had to be managed through careful deployment practice rather than through the framework itself. ROS 2 addressed the timing issue more directly by adopting the Data Distribution Service (DDS) and exposing Quality of Service (QoS) policies that map onto robotic timing and reliability requirements. A single QoS setting can, for example, mark a high-rate sensor stream as best-effort while keeping a command channel reliable [26]. Even so, empirical work shows that tail latency depends not only on network conditions but also on executor design and callback scheduling decisions within the runtime [11, 23]. Achieving predictable behavior requires end-to-end discipline across memory allocation, inter-process communication configuration, and OS kernel setup, with measurements taken to identify exactly where variance enters the system [29, 36].

The same end-to-end perspective applies below the middleware layer. In industrial networks, Time-Sensitive Networking (TSN) is often seen as a path to bounded-latency communication, yet measurement studies show that its guarantees can be undermined by integration choices and cross-traffic in real deployments [55].

Two existing approaches are worth distinguishing from TIC. AUTOSAR Timing Extensions provide a timing contract model for automotive ECU development with runtime monitoring of timing constraints [5]. In principle, one could imagine adding “uncertainty” and “drift” signals to an AUTOSAR timing chain, but this would still leave key gaps for learning-enabled robots: (i) AUTOSAR timing chains focus on deterministic execution/communication semantics, while learned components require explicit calibration/drift evidence (Section 4.6.2); (ii) the standard does not define how probabilistic evidence links to an executable safety-degradation policy; and (iii) robotics stacks commonly span heterogeneous compute (SoC+GPU), middleware (DDS), and edge/cloud paths that sit outside the traditional in-vehicle ECU scope.

ROSRV provides runtime verification for ROS by monitoring message-level safety properties via formal specifications [21], but operates primarily at the communication protocol/property level and does not treat timing envelopes, uncertainty calibration, drift monitoring, and scalability constraints as a single auditable contract artifact.

Table 1 summarizes the gap: TIC is not "yet another timing budget format," but a cross-layer contract that binds timing to learning-specific evidence and to an explicit fallback state machine.

Worked comparison (Option A): ROSRV vs. TIC on one AMR path. To make the comparison concrete, consider the warehouse AMR perception path used throughout this paper: *camera frame publish* $\rightarrow$ *detections publish* with a 30 Hz period and a 25 ms deadline. In ROSRV, the dominant artifact is a *message-level property* over the ROS communication graph (e.g., a temporal logic assertion that a detection message should follow each camera message within a bounded time window, and that certain fields satisfy application invariants). ROSRV therefore naturally expresses *ordering and safety properties* over topic streams, and can be used to raise alarms or trigger a property-driven response.

TIC adds three elements that are not first-class in ROSRV-style specifications: (i) a *latency envelope* expressed as percentiles (p50/p95/p99/max) under declared stressors, not only a single "within Δ" bound; (ii) *learning evidence* (calibrated uncertainty and drift metrics) that gates trust in detections; and (iii) an *executable fallback state machine* that binds monitoring outcomes to a tiered safety envelope (e.g., Tier 3 implies a safe-stop-compatible speed/stop constraint). The practical distinction is that ROSRV can state and monitor properties, while TIC couples timing evidence, learning evidence, and degraded-mode behavior into a single auditable contract document plus a runtime status stream.

2.4 Edge Intelligence and Cloud Robotics

Deciding where to execute computation, on the robot, on nearby edge infrastructure, or in the cloud, is no longer straightforward as model complexity and deployment ambition continues to grow. Offloading perception to the cloud provides access to larger computing resources, but it introduces round-trip latency and availability dependence that safety-critical loops cannot tolerate [24]. Edge and fog layers sit between the device and cloud, allowing local safety controllers to run with bounded latency while fleet analytics and model training are handled at remote infrastructure [22, 40]. Allocating work across this spectrum is an active research area, particularly as private wireless networks and 5G campus deployments continue to change the latency profiles of off-robot execution [32, 49]. Recent surveys and experimental studies argue that the central systems question is not "where is computing cheaper?" but rather "where can deadlines be guaranteed under interference and failure?" This re-framing, which motivates the TIC contract model introduced in Section 4.6, is consistent

Table 1: Clause-level comparison of TIC vs. existing timing/monitoring approaches. "Native" means the concept is explicitly modeled and intended for runtime checking; "Partial" means it can be approximated with custom signals/properties but is not first-class.

Capability	AUTOSAR TIMEX	ROSRV	ROS-Monitoring	ROS 2 Lifecycle	TIC (present work)
End-to-end timing chains / deadlines	Native [5]	Partial	Partial [15]	Partial	Native (latency envelope)
Runtime checking hooks	Native [5]	Native [21]	Native [15]	Native (lifecycle transitions)	Native (status stream + lifecycle binding)
Probabilistic uncertainty calibration (ECE, etc.)	No	No	No	No	Native (uncertainty envelope)
Distributional drift monitoring (PSI/MMD/ADWIN, etc.)	No	No	No	No	Native (drift envelope)
Executable safety fallback state machine (tier-down/stop)	Partial (platform-specific)	Partial (property-triggered)	Partial (property-triggered)	Partial (state mgmt only)	Native (fallback clause)
Observability/ telemetry contract (required metrics, rate)	Partial	Partial	Partial	Partial	Native (observability payload)
Scalability limits (rate/queue/fan-out)	Partial	No	No	No	Native (scalability clause)
Robotics middleware fit (ROS 2/DDS, edge/cloud paths)	Limited	ROS-specific	ROS-specific	ROS 2	ROS 2/DDS-oriented + edge-aware

with emerging contract-style edge robotics architectures [30]. Digital twins, continuously updated computational models of physical systems, have become a practical tool for embedding decision-making in a richer world model and for detecting anomalies and supporting predictive maintenance at fleet scale.

2.5 Learning in the Loop

Integrating learning into deployed robotic systems introduces new workload characteristics that classical real-time schedulability theory did not anticipate. Neural network inference brings several of these: input-dependent execution time, GPU memory pressure that varies with batch size and model complexity, and gradual distributional drift that can degrade prediction quality in ways

that are difficult to detect without explicit monitoring. Structured pruning, quantization, and knowledge distillation can meaningfully reduce inference latency [45], though the robustness of compressed models under distributional shift and corner-case inputs remains a concern that compression alone does not resolve [9, 38]. Beyond compression, calibrated uncertainty estimation and post-hoc calibration methods provide confidence signals that safety-monitoring layers can use directly [16, 20]. TinyML-style compression and deployment techniques can further reduce latency and power on microcontroller-class hardware. At the other end of the scale, large foundation models and vision-language-action architectures are reshaping how robotic capabilities are learned and assembled. Deploying these models within real-time latency constraints remains, for the most part, an open problem.

2.6 Synthesis: the missing joint artifact

Across the four bodies of work surveyed above—real-time scheduling, middleware/QoS, edge–cloud architecture, and learning-enabled autonomy—a common gap emerges: each contributes mechanisms for one dimension of the RT–ML integration problem, but none provides a single artifact that jointly binds timing evidence, learning-quality evidence, and fallback behavior into a runtime-auditable contract. TIC is designed to close this specific gap by making those obligations explicit, versioned, and monitorable.

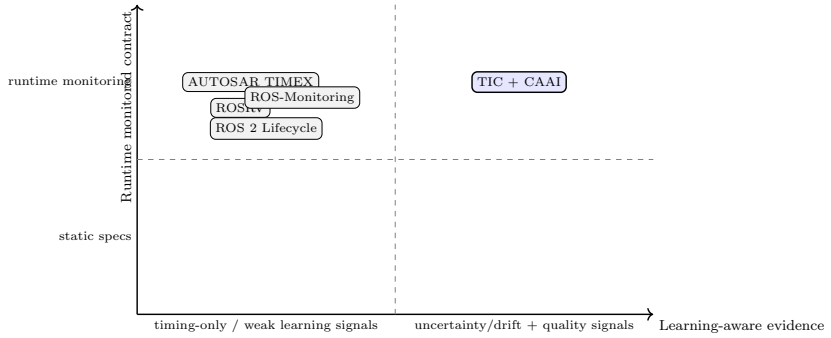

Fig. 1: A minimal taxonomy of contract approaches by (i) whether the artifact is designed for runtime monitoring and (ii) whether it is learning-aware (uncertainty/drift/quality evidence). TIC positions itself as a joint, runtime-auditable contract spanning both axes.

3 Real-Time Intelligence: Models, Metrics, and Design Contracts

3.1 Deadlines, Worst-Case Guarantees, and Timing Categories

Engineers sometimes describe a system as "real-time" when they mean it is fast. In safety-critical robotics, that shortcut is dangerous. A fast system has a low *average* response time; a real-time system has a *worst-case* response time that it can guarantee. The distinction matters because physical processes offer only one opportunity per cycle; miss the deadline and action is no longer possible.

The pacemaker analogy captures this well. It does not need to be fast in the benchmarking sense; it needs to be punctual in the physiological sense. Punctuality in the presence of interference and corner cases is the central obligation of real-time engineering.

Robotic stacks rarely fall into a single timing category. Some components are hard real-time, with catastrophic consequences if deadlines are missed, such as low-level torque control or safety interlocks. Others are soft real-time, where lateness reduces quality rather than causing immediate danger, such as a tracker that drops an occasional frame, with the state estimator filling the gap. The trap is to treat a soft real-time pipeline as sufficient and feed its delayed outputs into a hard real-time path without recognizing the timing dependency. When that happens, the control layer absorbs latency variance it was never designed to tolerate, and the safety case breaks down.

3.2 Latency Budgets End-to-End

Hard deadlines shift the focus to where the time actually goes. A latency budget makes that explicit: sensing, preprocessing, inference, estimation, planning, communication, and actuation each get a defined time allocation, and those allocations are engineering commitments, not measurements taken after the fact.

Table 2 breaks the 20 ms cycle of a 50 Hz control loop into its constituent stages, from sensing through actuation, with a jitter margin held in reserve. These numbers will shift with sensor suite, network topology, and compute platform, but one observation stays consistent: perception, once a small slice of the cycle, now takes up most of it, leaving little room for jitter.

That observation changes what "optimization" means in practice. It is rarely addressed by a single code change. Instead, it drives platform selection, hardware-software co-design, model compression, and in some cases adaptive inference strategies such as the contract-aware tier switching described in Section 5.4. These numbers should always be validated on the target deployment device, as timing characteristics vary significantly across hardware platforms.

Table 2: Example latency budget for a 50 Hz low-level control loop with a 20 ms period (values in ms).

Stage	Typical (ms)	Notes
Sensing capture + transfer	2-6	Camera exposure, IMU, encoder latency
Preprocessing	1-3	Rectification, downsampling
Perception / inference	2-10	Depends on model and hardware
State estimation	1-4	EKF/UKF, factor graphs (often async)
Planning	2-15	May run slower thread; uses predictions
Control synthesis	0.5-2	MPC, PID, whole-body control
Actuation command + bus	1-4	Motor drives, safety interlocks
Margin / jitter buf.	≥ 2	Required for worst-case conditions

Note: Ranges are not additive at their upper bounds; worst-case values for perception and planning do not co-occur under normal operating conditions. The budget applies to the end-to-end critical path, not the sum of all maximums (see Fig. 2 for a visual breakdown).

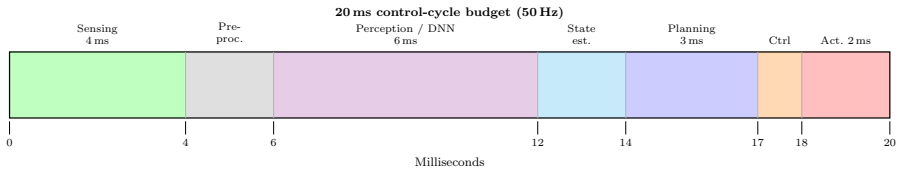

Fig. 2: Visual breakdown of the 20 ms (50 Hz) control-cycle budget. Bar width is proportional to time. The **Perception/DNN** segment (violet) dominates; **Actuation** (red) must always be guaranteed. (Author-created figure.)

3.3 Jitter, Determinism, and Priority

Average latency hides the behavior that most often breaks controllers: variation. Jitter, cycle-to-cycle variance in completion time, matters because many control laws implicitly assume a fixed sampling period. A controller tuned for a 1 ms loop may remain stable at 1 ms on average and still perform poorly if the period swings between 0.8 ms and 1.4 ms.

Mitigating jitter is less glamorous than designing a new planner, but it is typically more decisive for safety. In practice, teams rely on priority-based scheduling, explicit CPU isolation for time-critical threads, careful memory allocation to avoid page faults and cache thrash, and runtime choices that avoid unpredictable garbage collection on hot paths [10]. The challenge sharpens on heterogeneous SoCs: accelerators introduce their own scheduling queues and memory-bus contention, so variance on the GPU or NPU can propagate back

into CPU-side threads unless the whole platform is treated as a coupled timing system [11, 45].

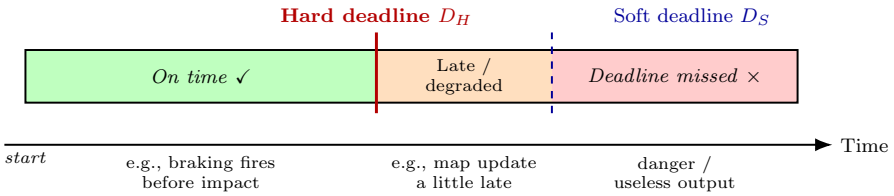

Fig. 3: Hard vs. soft real-time deadlines. **Hard deadline** D_H (solid red): missing it causes unsafe outcomes. **Soft deadline** D_S (dashed blue): late results degrade quality but remain safe. The red zone is always unacceptable regardless of task type. (Author-created figure.)

3.4 Synchronization in Multi-Sensor Fusion

Multi-sensor fusion looks simple: "it combines measurements to get a better estimate". The difficulty, as teams discover in practice, is temporal. Sensors operate on separate clock domains, their acquisition pipelines have different latencies, and networked transport introduces variable delay. A fusion system that ignores these facts often produces estimates that are numerically plausible but temporally misaligned.

Three broad approaches have emerged in practice. Hardware timestamping pushes time attribution as close to acquisition as possible. Asynchronous estimators accept out-of-sequence measurements and fuse them with explicit time models, often in Kalman-style frameworks [46]. Scheduling and observability choices can be used to prioritize sensor streams whose loss or delay would most strongly degrade safety-critical state estimates [28].

Strict sensor alignment increases sensor-to-actuator delay, whereas accepting irregular arrival reduces delay but can introduce cross-modal inconsistency. The right choice depends on what downstream controllers can tolerate and how much slack the end-to-end latency budget provides.

Vision-centric autonomy: a case study in acceptable failure modes

Most high-integrity robotic stacks use redundant sensing across dissimilar modalities, typically cameras, radar, and LiDAR, because heterogeneity reduces the probability that a single environmental condition disables the full perception chain. Recent surveys of autonomous-vehicle sensing emphasize that sensor selection goes beyond performance optimization: it is a design decision that shapes the system's cost structure, operational envelope, and safety requirements [51, 53][1].

A contrasting philosophy is to minimize the sensor suite and rely on large-scale learning to compensate for reduced geometric redundancy. On the surface this

[1] Reference [53] was in press at the time of this revision (April 2026).

is attractive: fewer sensors simplify calibration and integration, and the bill of materials drops. The technical cost is subtler. With fewer independent physical cues, residual risk shifts toward the learned perception model's behavior under rare weather conditions, unusual illumination, partial occlusion, or distributional shift. Empirically, the robustness benefits of complementary modalities are visible in recent multi-modal fusion results, where camera-LiDAR-radar combinations improve generalization across conditions compared to single-modality baseline [1].
For system designers, the practical takeaway is straightforward: a vision-centric stack must compensate with stronger runtime monitoring, explicit uncertainty handling, and conservative fallback behavior, and those compensations must be justified with evidence in the safety case.

3.5 Real-Time Machine Learning: Throughput vs. Worst Case

Machine-learning models are often evaluated as if their cost is a single number: an average latency on a benchmark and an average error on a dataset. Real robots do not experience averages. They experience tails: the occasional cluttered frame, the unusual lighting condition, the burst of competing load that pushes inference from "usually fine" into "occasionally late."

This is why worst-case or tail latency matters as much as mean throughput. A model that runs in 12 ms at the 95th percentile but stretches to 45 ms in rare cases may be acceptable for a soft real-time perception channel and simultaneously unacceptable for any path that influences a hard real-time safety envelope. Designing for the tail becomes a distribution-shaping problem rather than a mean-shifting one.

Several families of techniques help compress the tail. Early-exit architectures can terminate computation when intermediate confidence is already sufficient. Cascades route easy inputs through lightweight classifiers and reserve heavy models for the ambiguous cases. Fixed-precision compilation and careful runtime configuration reduce numerical and memory-allocation variance, and hardware-aware scheduling disciplines such as batch size fixed to one, pre-allocation in pinned regions, reduce contention. Well-specified fallback policies matter: when a primary model misses its budget, the system must have a safe substitute that preserves the contract rather than silently violating it [3, 45].

3.6 Why Combining AI and Real-Time Is Structurally Difficult

Real-time engineering and modern AI grew up with different success criteria. In a real-time context, the question is whether the system will meet its deadline under stress every time; in much of machine learning, the question is how the average-case error behaves on a benchmark. When these worldviews meet within a deployed robot, the mismatch manifests as a collection of "small" effects that interact and occasionally coalesce into a large failure.

The most obvious friction is execution-time variability. The same network can finish quickly on an easy input and stall on a cluttered one, and that variability becomes harder to bound as models become deeper and more dynamic.

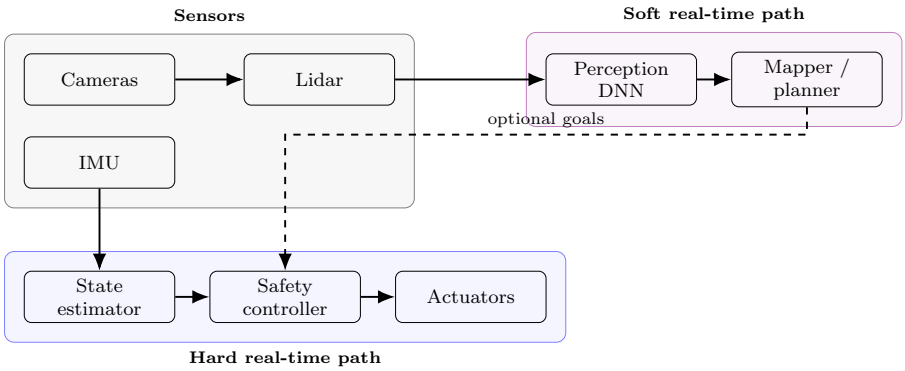

Fig. 4: Dual-path architecture: a hard real-time safety loop (state estimation + safety controller) runs with bounded latency; perception and mapping run asynchronously with watchdog supervision and conservative fallbacks. (Author-created figure.)

Worst-case inference latency for transformer-style architectures remains an open research problem, precisely because attention and memory traffic are sensitive to input structure and runtime contention [3, 35].

Less visible, but operationally important is numerical non-determinism. Parallel GPU kernels can change the order of floating-point reductions, producing outputs that are close but not identical. For perception, this is usually harmless; for safety monitors implemented as threshold tests, it can create problematic edge cases where a signal hovers around the boundary.

The harder problem is that these effects rarely occur in isolation. A perception pipeline shares caches, memory bandwidth, and sometimes even power and thermal headroom with control and communication threads. Under load, contention-driven jitter can propagate into the very layer that must stay deterministic, unless cores, memory, and execution budgets are deliberately partitioned.

Learned components are also difficult to justify to external assurance processes. When a controller is specified as code plus a proof obligation, failures can be traced to violated assumptions. When behavior is embodied in weights, the same diagnostic story is harder to tell, yet regulators increasingly expect such structured explanations [9, 38].

3.7 Worked Example: Temporal Layers in an Autonomous Vehicle

Timescale separation is how a well-engineered autonomous vehicle handles these tensions. At the lowest layer, running at 1 000 Hz with microsecond deadlines, brake and steering controllers read wheel-speed sensors and adjust hydraulic pressure; these loops are implemented on dedicated real-time hardware and intentionally isolated from the software stacks above them. At 100 Hz, a sensor

fusion layer integrates camera frames, LiDAR, radar, and GPS into a unified world picture with hard real-time guarantees on its own output latency. At 10 Hz, a prediction and motion planning layer evaluates candidate paths given the world state from fusion; planning uses predictive models with a longer compute budget because the output is consumed asynchronously by the control layer. At roughly 1 Hz, a behavioral planner makes slower maneuver and route decisions. Asynchronously in the background, a cloud connection downloads updated map tiles and improved model weights.

The structural key is that no layer waits for the layer above it. If the behavioral planner consumes an extra 500 ms, the motor controllers and sensor fusion layer continue operating correctly because they hold the most recent valid goal and execute against it. This non-blocking, timescale-separated hierarchy is the canonical pattern for real-time intelligence in mobile autonomous systems, and its integrity depends on lower layers having fallback behaviors that remain safe in the absence of fresher higher-level guidance.

4 Scalable Architecture for Robotics

Robotic scalability spans at least four independent dimensions: compute capacity per robot, communication bandwidth and latency within and across robots, organizational modularity that allows teams to evolve subsystems independently, and verification coverage that does not collapse as system complexity grows. A design that works well for a single robot in a controlled lab tends to fail in at least five observable ways as deployment scope expands, each requiring deliberate architectural decisions rather than incremental performance tuning.

4.1 Five Dimensions of Growth

Robotic systems "scale" along more than one axis, and the axes interact. A design change that appears harmless in a single-robot prototype can become the dominant bottleneck once the same code is replicated across a fleet.

One axis is *sensor complexity*. Moving from a single camera to a multi-modal suite, cameras, inertial sensors, and ranging, increase not only bandwidth but also synchronization work and fusion latency. A second axis is *task complexity*. Point-to-point navigation assumes relatively stable environments, whereas operation in human spaces forces planners to account for uncertainty, congestion, and social constraints, all of which increase the state space and replanning frequency.

A third axis is *fleet scale*. Coordination overhead grows nonlinearly because communication, shared-map consistency, and contention for shared infrastructure such as SLAM services or dispatch servers, introduce tail-latency effects that do not appear in small-team tests. The remaining two axes are tied to learning. First, *computational demand* has risen with modern perception models, making the robot's thermal and power envelope a real-time consideration.

Second, *geographic scope* introduces wide-area links and intermittent connectivity, which means the architecture must degrade gracefully. Local safety loops must remain deterministic even when global services are slow or unreachable.

4.2 Layered and Hierarchical Control

A common architectural answer to the "fast versus smart" tension is to separate functions by timescale. Hierarchical control isolates the reactive safety layer, the part that must respond within milliseconds and can be analyzed as hard real-time, from deliberative layers that optimize over seconds or minutes and can therefore tolerate longer compute budgets [8].

Brooks's subsumption architecture offered an early, influential expression of this principle. Modern stacks look different in detail, but they preserve the same invariant: overload or failure in a higher layer must not be able to stall, starve, or otherwise compromise the guarantees of the lowest safety-critical layer. When that invariant is violated, architectural complexity turns directly into timing risk.

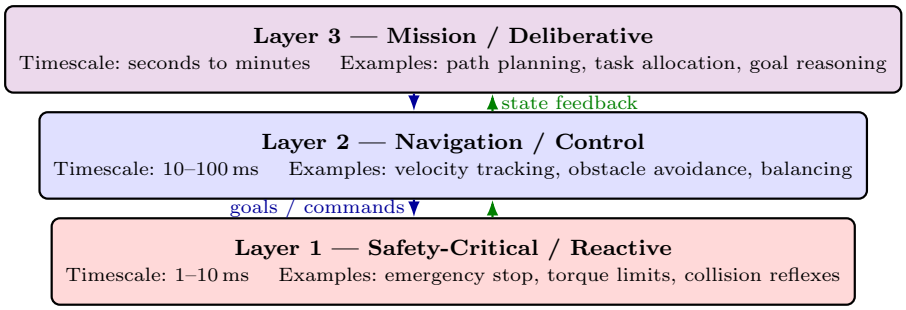

Fig. 5: Three-layer hierarchical control architecture. Layer 1 (red) provides hard real-time guarantees: it responds within 1–10 ms regardless of the higher layers. Goals flow *downward* (solid); state feedback flows *upward* (dashed). If a higher layer crashes, lower layers continue to guarantee basic safety. (Author-created figure.)

4.3 Middleware, QoS, and Discovery

Middleware is where architectural intent meets transport reality. Publish-subscribe abstractions only become useful when the runtime provides concrete controls over reliability, buffering, and delivery semantics. ROS 2 exposes these controls through DDS QoS policies tuned to robotic timing and reliability requirements [23, 26].

In this context, "more reliable" is not always "better." A high-rate camera stream often benefits from best-effort delivery with volatile durability: dropping an occasional frame is typically preferable to back-pressure that inflates latency. Command and safety channels invert that trade-off. If a velocity command is lost, the robot can enter an indeterminate state, so reliable delivery and careful queue sizing become part of the safety argument. Table 3 collects representative combinations that recur in deployed systems.

Table 3: Common ROS 2 QoS policy combinations and recommended use cases.

Channel type	Reliability	Durability	Typical use case
Sensor stream (camera, lidar)	Best-effort	Volatile	High-rate data; Occasional dropped frame is acceptable
Velocity / command	Reliable	Volatile	Must not lose messages; delivery order matters
Parameters / config	Reliable	Transient-local	Late-joining nodes receive the last stored value
Diagnostics / health	Best-effort	Volatile	Lightweight; occasional loss acceptable
Map / occupancy grid	Reliable	Transient-local	Large payload; new subscribers need the current map

4.4 Horizontal Scaling: Multi-Robot and Swarms

Fleet coordination reveals scaling limits quickly. Once the number of robots grows, a single planner is asked to do two incompatible jobs: maintain global situational awareness and respond within tight latency bounds. At that point, centralization turns into both a bottleneck and a single point of failure.

Partially distributed approaches give up some global optimality at the expense of throughput and robustness. Market-based allocation, distributed MPC, and consensus protocols represent different points in that trade space [18]. In warehouse-style settings, auction mechanisms are attractive because their per-task computation can scale well and because they gracefully degrade as robots join, leave, or fail.

Coordination is not "just an algorithm" layer: it is an architectural choice that shapes tail latency under load. Table 4 summarizes these paradigms with an explicit focus on those trade-offs.

4.5 Vertical Scaling: From Embedded to Cloud

Vertical scaling concerns what the robot can do with the compute and power it carries. For many platforms, perception is the limiting workload: models

Table 4: Comparison of coordination paradigms (qualitative).

Paradigm	Strengths	Risks / costs
Centralized planner	Global optimality (small teams)	Single point of failure
Hierarchical	Clear command structure	Interface rigidity
Distributed consensus	Robustness; no central brain	Slower convergence
Learning-based	Handles messy environments	Data hunger; hard to verify

must deliver adequate accuracy while staying inside tight thermal and battery envelopes. As transformer-based architectures have increased computational demand, the gap between cloud-scale training hardware and edge deployment hardware has widened, and performance now depends on co-design choices such as quantization, structured sparsity, and operator fusion [45].

Heterogeneous SoCs address this by combining CPUs, GPUs, and neural accelerators into a single package. Used well, an accelerator can provide lower variance than general-purpose GPU execution because it reduces contention and constrains the runtime behavior more tightly. Used poorly, it simply adds another queue and another point of contention.

Table 5 summarizes the practical trade-offs when these design choices are pushed across the edge–fog–cloud tiers.

Table 5: Edge–cloud deployment trade-offs.

Dimension	On-robot	Fog / on-prem	Cloud
Typical latency	< 5 ms	5–50 ms	50–500 ms
Compute power	Low – medium	Medium – high	Very high
Connectivity needed	None	Local network	Internet
Best suited for	Safety reactions	Fleet coordination	Model training
Main risk	Limited RAM/GPU	LAN outage	High delay

4.6 Temporal Intelligence Contract (TIC)

Existing approaches to system contracts in distributed software, service level agreements, interface description languages, and component-based design notations, handle functional correctness and availability well but leave timing, uncertainty, and safety-specific degradation behaviors unaddressed. This work introduces the **Temporal Intelligence Contract (TIC)**, a reference-specification that goes beyond static documentation to one that can be audited and automatically monitored at runtime.

A TIC is a machine-readable contract that holds each component to five obligations. The **latency envelope** specifies best-case, typical, and worst-case completion times on the actual deployment hardware, measured under realistic concurrent load, not under ideally isolated conditions. The **uncertainty envelope** defines calibration requirements and the maximum distributional drift the system will tolerate before downstream safety logic stops trusting the component's predictions. The **safety fallback policy** prescribes degraded-mode and emergency-stop behaviors triggered when other clauses are violated, captured as a formal state machine that operates without human intervention. The **observability payload** defines the traces, timing counters, and health probes the component must emit continuously for contract verification during operation. The **scalability clause** caps fan-out, topic publication rates, and admission-control behaviors under load, preventing a misbehaving component from propagating back-pressure through the system.

Unlike a static requirement specification, TIC treats its obligations as *operational claims* that must remain true at runtime. That orientation changes how teams build systems. Instead of treating timing and safety as properties established once during integration testing, TIC assumes conditions will drift, through thermal throttling, network contention, sensor degradation, and workload changes, and requires the system to detect drift early enough to stay safe.

Consider a warehouse AMR whose local perception module publishes obstacle detections at 30 Hz. A classical interface description would specify message fields and perhaps an update rate. A TIC also binds the module to (i) a worst-case end-to-end detection latency on the target SoC, (ii) an uncertainty calibration requirement, such as confidence scores remaining calibrated under the site's lighting conditions, and (iii) a specific degraded behavior when those claims stop being true, such as reducing speed, widening stop distances, or entering a supervised mode. The fallback is not informal "operator will handle it" assumption; it is part of the machine-executable contract. A complete structured schema and JSON schema for TIC clause instantiation is beyond the scope of this paper; the clause structure in Table 8 defines the required fields and types sufficient for conforming implementation.

Runtime monitoring also changes failure analysis. When an incident occurs, TIC-style telemetry provides a structured record of whether deadlines were being approached, whether uncertainty was drifting, and whether back-pressure was building in the communication graph. This gives the missing connective tissue between learning-system behavior and the evidence that a safety case demands.

The violation penalty term P_v is defined as the ratio of contract-violation events to total monitored cycles: $P_v = N_{\text{violations}}/N_{\text{cycles}}$; for the example in Table 6, $P_v = 3/60 = 0.05$.

The RTI-Score provides a deployment-readiness metric that aggregates compliance across clauses:

$$\text{RTI-Score} = \min\Big(S_s,\ w_1 S_t + w_2 S_q + w_4 S_o\Big) - w_5 P_v, \quad \text{subject to } S_s = 1.0$$

This conjunctive form encodes a safety-engineering first principle: *non-safety terms must not compensate for a safety violation.* If safety compliance drops below 1.0 (fallback triggered or safety invariant violated), the score collapses regardless of timing, quality, or observability. The weights w_1, w_2, w_4, w_5 are deployment-specific and must be calibrated through hazard analysis (FMEA/HAZOP) consistent with the applicable safety standard (ISO 26262, IEC 61508, or ISO 10218).

NOTE: The weights $w_1 = 0.15$, $w_2 = 0.15$, $w_3 = 0.35$, $w_4 = 0.10$, $w_5 = 0.25$ below demonstrate arithmetic only. Production deployments *must* derive w_i from system-level hazard analysis (FMEA or HAZOP) against the applicable safety standard (ISO 26262, IEC 61508, or ISO 10218).

Table 6: Worked example of RTI-Score computation for an AMR perception module over a 60-cycle monitoring window.

Term	Value	Rationale
S_t (timing compliance)	0.92	92% of deadlines met in window
S_q (task quality)	0.87	Detection F1 = 0.87 on deployment distribution
S_s (safety compliance)	1.00	No safety-fallback triggers in window
S_o (observability)	0.78	22% of expected telemetry traces missing
P_v (violation penalty)	0.05	3 minor contract violations in window

With $w_1 = 0.15$, $w_2 = 0.15$, $w_3 = 0.35$, $w_4 = 0.10$, $w_5 = 0.25$: RTI-Score $= \min(1.0, 0.138 + 0.131 + 0.078) - 0.013 = 0.334$.

Weights w_i are application-specific and must be determined during system-level hazard analysis, such as FMEA or HAZOP, consistent with safety standards such as ISO 26262, IEC 61508, or ISO 10218; w_3 and w_5 typically carry the highest values because safety compliance and violation penalties take priority over throughput. A module whose RTI-Score drops below a deployment-specified threshold can be quarantined for investigation without bringing down the wider system.

4.6.1 Worked FMEA grounding for RTI-Score weights

To reduce "numerology" risk in weight selection, a minimal Failure Modes and Effects Analysis (FMEA) can be used to map dominant hazards to TIC clauses and to derive weight priors. Table 7 illustrates this for a warehouse AMR perception pipeline. Each failure mode is scored by Severity (S), Occurrence (O), and Detectability (D) on a 1–10 scale, producing a Risk Priority Number (RPN $= S \times O \times D$). RPN mass is then allocated to TIC clause families;

weights w_i are set proportional to those allocations (and must be reviewed by the responsible safety process owner).

Table 7: Example FMEA for a warehouse AMR perception path and mapping to TIC clauses.

Failure mode	Effect	S	O	D	RPN	Primary TIC clause
Perception deadline misses under load	Late obstacle updates; unsafe braking margin	9	4	4	144	Latency envelope / penalty
Confidence miscalibration	False safe states; unsafe speed selection	9	3	6	162	Uncertainty envelope
Distributional drift (lighting/reflective floors)	Degraded detection quality; near-miss risk	8	4	5	160	Drift envelope / fallback
Fallback not triggered when needed	Unsafe continuation in violated state	10	2	6	120	Safety fallback
Missing telemetry/trace gaps	Inability to audit and diagnose violations	6	4	7	168	Observability payload
Queue growth/back-pressure cascade	Latency amplification across graph	7	3	5	105	Scalability clause

Weight derivation (example). Aggregating RPN by clause family yields a prioritization in which safety/fallback and violation penalty are dominant, followed by timing and observability, with task-quality weighted lower unless the mission is efficiency-critical. In practice, teams can normalize clause-RPN totals to define initial w_i values, then calibrate against field evidence and safety targets.

Table 8: TIC clauses for multi-tier robotic systems.

TIC clause	Required artifact	Runtime check	Typical owner
Latency envelope	Deadline profile per path	Deadline miss ratio	RT engineer
Uncertainty envelope	Calibration report, OOD threshold	Confidence drift	ML engineer
Safety fallback	State machine for degrade/stop	Trigger rate	Safety engineer
Observability payload	Trace schema, metrics contract	Missing telemetry	DevOps / SRE
Scalability clause	Rate limits, backpressure policy	Queue depth	Platform architect

4.6.2 Reference TIC schema and monitoring semantics

To make TIC implementable and auditable, a conforming component SHOULD publish a versioned contract document (JSON, schema version `1.0.0`) and a runtime "contract status" stream. The document defines static intent (thresholds, deadlines, ownership, and immutable `component.build` provenance); the status stream provides the time-indexed evidence used for compliance. A complete JSON Schema (draft-07) is maintained as `tic\_monitor/schema/tic.schema.json`.

Table 9: Compact TIC contract schema excerpt (required fields).

Field	Type	Meaning
tic_version	string	Contract schema version (semver, e.g., 1.0.0).
component.name	string	Component identifier (node/service).
component.build.type	string	Provenance kind (`git`, `container`, `semver`).
component.build.id	string	Immutable identifier (commit hash, digest, or release tag).
component.build.image	string	Optional OCI image reference when `type=container`.
time_base	string	Clock source for timestamps (e.g., monotonic).
paths[*].deadline_ms	number	Deadline per monitored end-to-end path.
paths[*].latency_env_ms	object	p50/p95/p99/max under declared conditions.
uncertainty.cal_metric	string	Calibration metric name (e.g., ECE).
uncertainty.drift_metric	string	Drift metric name (e.g., PSI, MMD).
fallback.state_machine	string	Named fallback policy/state machine.
observability.metrics	array	Metrics/traces the component must emit.
scalability.publish_rate_max	number	Declared rate ceiling to prevent back-pressure.

Minimal TIC contract example (JSON)

```
{
  "tic_version": "1.0.0",
  "component": {
    "name": "perception_node",
    "build": {
      "type": "git",
      "id": "abc123def4567890",
      "image": "registry.example/perception:1.2.3"
    }
  },
  "critical_paths": [
    {
```

```
        "path_id": "camera->detections",
        "deadline_ms": 25.0,
        "latency_envelope_ms": {
          "p50": 8.0,
          "p95": 14.0,
          "p99": 20.0,
          "max": 25.0
        }
      }
    ],
    "uncertainty": {
      "calibration_metric": "ECE",
      "ece_max": 0.05,
      "drift_metric": "PSI",
      "drift_warn": 0.20,
      "drift_violate": 0.35
    },
    "fallback": {
      "state_machine": "TIER_DOWN_OR_STOP",
      "max_consecutive_deadline_misses": 2
    },
    "observability": {
      "required_metrics": ["latency_hist", "deadline_miss", "queue_depth"],
      "min_emit_hz": 1.0
    },
    "scalability": {
      "publish_rate_hz_max": 30,
      "queue_depth_max": 10,
      "backpressure_policy": "drop_oldest"
    }
  }
```


Monitoring semantics (required). A TIC monitor MUST specify: (i) window definition (e.g., last N cycles or T seconds), (ii) the exact test for "deadline miss" (e.g., end-to-end timestamping boundaries), (iii) how uncertainty and drift metrics are computed and rate-limited, and (iv) how violations are aggregated into triggers for the fallback state machine. For example, Expected Calibration Error (ECE) is a standard calibration summary metric [20]. Population Stability Index (PSI), commonly used in credit risk monitoring, is one simple drift indicator [42]; alternatives include kernel MMD [19] and streaming change detectors such as ADWIN [7].

Conformance checklist (minimum). A TIC-conforming component MUST: (1) expose deadlines/paths with units; (2) emit per-window miss statistics; (3) emit the uncertainty/drift signals named in the contract; (4) expose the current fallback state; and (5) enforce the declared scalability limits (rate/queue/fan-out) locally, not as a best-effort operator procedure.

4.7 Worked Example: Scaling a Warehouse Robot Fleet

A warehouse fleet provides a revealing stress test because scaling tends to happen in clear inflection points. Imagine a team that starts with five robots and grows to two hundred over a few years. Early on, a flat task-assignment

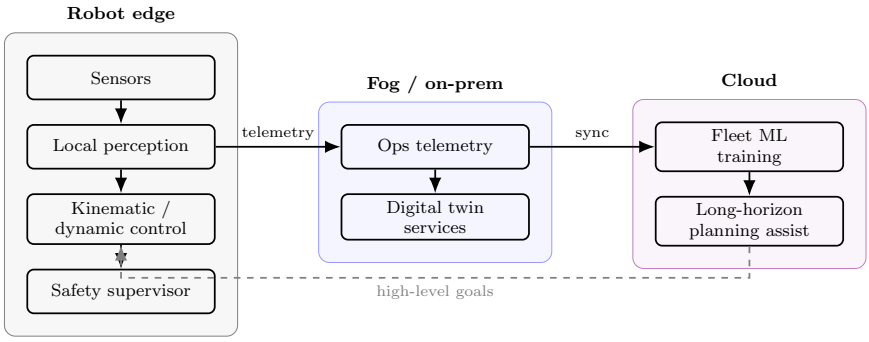

Fig. 6: Typical edge-cloud split: closed-loop safety and actuation remain on-robot; fleet analytics and training reside off-robot with asynchronous interaction. Solid arrows: real-time data flow. Dashed arrow: asynchronous goal delivery from cloud planner to on-robot kinematic controller.

module may appear sufficient because the system rarely experiences contention. As the fleet approaches a few dozen units, however, that same module becomes a latency bottleneck; replacing it with an auction-style allocator restores assignment responsiveness without forcing changes elsewhere, provided the interface boundaries were designed well [52].

Shared perception infrastructure typically becomes the next constraint. When each robot streams sensor data to a central SLAM service, the service can saturate before any individual robot appears overloaded. Splitting SLAM into per-zone edge instances and synchronizing map segments asynchronously reduces tail latency and prevents congestion in one zone from propagating across the fleet.

Outages then test whether the system is resilient or merely fast. When a network partition isolates a warehouse zone, robots must fall back to local navigation with conservative obstacle avoidance. Across all of these transitions, the core discipline is separation of concerns: the 1 000 Hz motor controller and safety interlocks remain stable regardless of what happens in task allocation and fleet coordination above them.

5 Integrating Learning Without Breaking Time

5.1 Model Lifecycle and Real-Time Inference

A production learning pipeline for robotics extends well beyond training. It includes how data is collected and governed, how models are validated against representative operational conditions, how releases are staged, and how rollbacks are executed when field performance drifts.

What makes robotics distinct is that learning runs in a closed loop. Predictions change actions; actions change what the robot sees next; and the

logged data that feeds the next training cycle is therefore shaped by the current model's biases [37]. A navigation policy that is slightly over-confident in one corner case can steer the robot into trajectories that over-represent that corner case, slowly shifting the training distribution away from the scenarios that the system must handle safely.

For that reason, monitoring is not a "nice-to-have" add-on. Detecting distributional shift early, before it accumulates into a safety-relevant degradation, requires instrumentation that is designed in and maintained over time. This is exactly the motivation behind the TIC uncertainty-envelope and observability clauses: they turn monitoring expectations into explicit obligations rather than best-effort practice.

5.2 Fleet-Scale Learning

Fleet-scale deployment turns learning into an operational systems problem. Once a robotic product runs as a fleet, it starts to encounter the long tail of rare conditions at a pace that no lab campaign can match. The implication is that model improvement depends as much on the data and evaluation pipeline as on the model class itself. Recent work on shared fleet learning emphasizes both the benefits of aggregated experience and the constraints it introduces, including privacy, annotation cost, governance, and validation burden [54]. Large-scale autonomous driving datasets further demonstrate the value of aggregated experience for learning-based perception in complex environments [44].

Fleet learning must be designed as part of the real-time system. Logging budgets, data transport, and release mechanisms are not free; they compete with on-device compute, storage, and bandwidth. TIC-style clauses provide a practical way to make those costs explicit: observability and scalability requirements can be stated, monitored, and enforced rather than assumed.

Fleet learning also introduces verification asymmetry. A modular pipeline can often be tested component-by-component with interpretable intermediate signals. End-to-end learning reduces hand-built interfaces, but it can also reduce debuggability when a failure occurs because the system offers fewer "hooks" for diagnosis. It pushes more of the assurance burden onto monitoring, regression test design, and evidence collection, and it makes safety case construction more demanding when the certified unit is an opaque network rather than a specified control law [9, 38]. In that sense, the modular and end-to-end approaches are not a simple quality ladder; they are different engineering bets with different evidence requirements and different failure modes.

5.3 Simulation and Digital Twins

Simulation has shifted from convenience to a necessity in modern robotics. When collecting real-world data is expensive, slow, or dangerous, a high-fidelity

simulator can generate repeatable conditions for debugging and for stress-testing policies against rare scenarios that would be irresponsible to stage physically [39].

The value of simulation, however, is bounded by the sim-to-real gap. Rendering and physics engines cannot perfectly reproduce sensor noise, contact dynamics, or the long-tail interactions that occur in deployed environments. Domain randomization and adaptation reduce this gap, but they do not eliminate it; the practical response is to treat simulation results as evidence that must be calibrated against field behavior using explicit validation protocols [47, 56].

Digital twins extend the idea beyond pre-deployment testing. Instead of being a standalone scenario generator, a twin becomes a continuously updated operational model used for monitoring, anomaly detection, and predictive maintenance at scale.

5.4 Contract-Aware Adaptive Inference (CAAI)

The TIC framework defines the obligations; CAAI is a concrete way to satisfy them in a perception stack that must run under changing conditions. The motivating observation is simple but easy to ignore during development: no single model choice maximizes accuracy, minimizes worst-case latency, and minimizes compute cost at the same time. On finite hardware, those objectives conflict, and the conflict is sharpest exactly when the robot is stressed, in crowded scenes, degraded sensing, network load, thermal throttling, or other competing activity. CAAI treats perception quality as a *managed resource*. Rather than locking in a single operating point, it runs a small family of models: a full-resolution model that provides the best quality when budget allows, a compressed model that reduces compute and variance, and an emergency tier whose outputs are minimal but whose completion time is compatible with the safety-critical budget. Conceptually this is aligned with anytime algorithms and imprecise computation models, where result quality is traded against time/compute under explicit control [12, 41, 57].

The key engineering choice is that switching in CAAI is predictable by design, rather than opportunistic. A monitor tracks three contract-relevant signals: the remaining latency margin including measured jitter, calibrated predictive uncertainty, and a drift indicator that estimates how far the current operating distribution has moved from training. Those signals are evaluated with hysteresis and minimum dwell times to avoid oscillation. In effect, tier-down transitions are treated as safety actions and are allowed to occur quickly; tier-up transitions require sustained evidence that the margin has recovered.

An automatic transmission captures the idea. In a clear aisle, the high-fidelity model can run comfortably within budget and deliver richer detections. When the robot enters a cluttered zone and margins shrink, the system shifts down *before* a hard deadline is violated. If uncertainty rises or deadline misses begin to accumulate, the contract requires the physical envelope to change as well, reducing speed, widening stop distances, or entering a protected mode,

so that stopping behavior remains consistent with the active perception tier. When conditions stabilize and contention drops, higher-quality inference can be restored without operator intervention.

The result is not adaptive inference as an isolated trick, but a closed-loop coupling between model choice and safety envelope. That coupling is what makes CAAI compatible with an auditable argument: the switching policy is driven by TIC variables, namely budget margin, uncertainty, drift and constrained by explicit invariants rather than by ad-hoc heuristics.

5.4.1 CAAI switching policy (reference state machine)

CAAI can be implemented as a small deterministic supervisor whose output is the active tier $\tau \in \{1, 2, 3\}$ and the corresponding safety envelope mode (e.g., speed cap, stop distance). A minimal reference policy is:

Reference CAAI policy (pseudocode)

```
Inputs each cycle/window:
  margin_ms      := deadline_ms - p99_latency_ms
  miss_rate      := deadline_misses / window_cycles
  unc            := calibrated uncertainty summary
                    (e.g., max over ROI)
  drift          := drift score (e.g., PSI)
State:
  tier in {1,2,3}, dwell_cycles
Parameters (from TIC):
  M_down12, M_up21, M_up11, U_23, U_32,
  D_warn, D_violate, dwell_min

If dwell_cycles < dwell_min:
  keep tier; dwell_cycles++
Else:
  // Fast degrade path (safety action)
  If (margin_ms < M_down12) or (miss_rate > 0):
    tier := max(tier, 2)
  If (unc > U_23) or (drift > D_violate)
     or (consecutive_misses >= 2):
    tier := 3

  // Conservative recovery path
  If (tier == 3) and (unc < U_32) and (drift < D_warn)
     and (margin_ms > M_up21):
    tier := 2
  If (tier == 2) and (unc is stable) and (drift < D_warn)
     and (margin_ms > M_up11):
     tier := 1

  dwell_cycles := 0
Output actions:
  publish active tier + safety envelope mode
  enforce tier-specific speed/stop limits

auditable invariants:
  tier 3 implies safe envelope
    (e.g., stop-distance >= worst-case braking distance)
  tier-up transitions require dwell_min cycles
    without misses
```

The two-threshold recovery design ($M_{\text{up11}} > M_{\text{up21}}$) prevents oscillation between Tier 2 and Tier 1 under marginal conditions; both values must be declared in the TIC contract document.

5.4.2 Distinction from anytime algorithms and cascades

Anytime algorithms, cascades, and early-exit policies treat adaptation primarily as a *compute* decision: given a time or energy budget, they select an inference depth or model to maximize expected accuracy [12, 41, 57]. In that framing, tier switching is an optimization knob and the safety controller is typically external (or implicit) to the inference policy.

CAAI's central difference is cross-layer: it couples tier selection to an explicit *safety envelope* transition declared in TIC. The supervisor's invariant is not "run the best model you can," but "the active tier implies an envelope that keeps the system safe under worst-case timing and evidence conditions." Concretely, Tier 3 is defined to be safe-stop-compatible (Section 5.4.1) and therefore forces a conservative control mode (e.g., reduced speed cap and increased stopping distance) when uncertainty/drift rise or timing margin collapses. This makes tier-down a safety action, not only a compute action, and it yields an auditable mechanism: the inference policy and the physical operating envelope move together as a single contract-governed transition.

Shielding and runtime monitoring are the closest prior patterns [2]: a shield enforces hard constraints regardless of what a learned policy proposes. CAAI complements that by addressing *which* learned component to run under varying timing/evidence conditions and by making the switching logic itself a declared, monitored contract artifact rather than an ad-hoc heuristic.

Signal and threshold definitions. In the reference policy, each decision is evaluated over a monitoring window of `window_cycles` control cycles (or messages) at the relevant rate. We track `consecutive_misses` as the number of back-to-back deadline misses within the current window (reset to zero on any on-time completion). "`unc is stable`" means the uncertainty summary remains below its recovery bound and does not exhibit upward trend over the last `dwell_min` cycles (e.g., bounded slope or variance). Threshold parameters are provided by the TIC: $M_\cdot$ are latency-margin thresholds in milliseconds; $U_\cdot$ are bounds on the chosen calibrated uncertainty summary (e.g., max ECE proxy or predictive entropy over ROI); and $D_\cdot$ are bounds on the chosen drift statistic (e.g., PSI/MMD/ADWIN score). Implementations must state the exact uncertainty and drift metrics, units/ranges, and windowing choices to make the switching behavior reproducible and auditable.

6 Reference Implementation Guidance and Reporting Protocol

This section outlines a minimal, reproducible prototype decomposition and a measurement/reporting protocol intended to make TIC and CAAI adoption auditable and comparable across implementations.

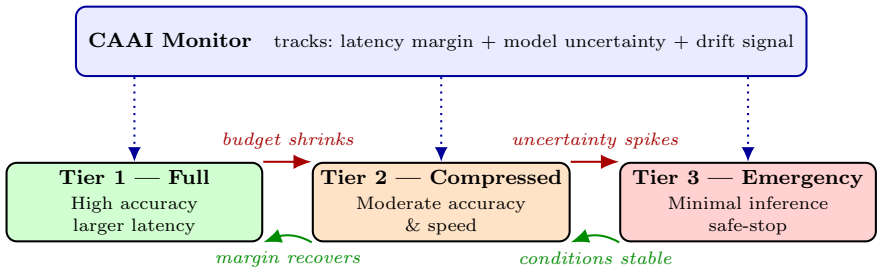

Fig. 7: Contract-Aware Adaptive Inference (CAAI). The **monitor** (blue, top) continuously tracks latency budget, model uncertainty, and drift, issuing vertical control signals (dotted blue arrows) to each tier. Red arrows (above): degradation path when conditions worsen. Green arrows (below): recovery path when conditions improve. The system maintains the highest model quality the environment allows at any given moment, without a hard failure. (Author-created figure.)

Table 10: Example mapping of algorithmic components to timing tiers.

Tier	Period / trigger	Examples	Failure if missed
Safety-critical	1–10 ms periodic	Torque limits, collision reflexes	Collision / harm
Control	10–100 ms	Balancing, tracking	Instability / damage
Perception	10–30 Hz aperiodic	Detection, segmentation	Poor decisions
Mapping	1–10 Hz	SLAM updates	Drift, wrong goals
Fleet ops	Seconds–minutes	Task assignment	Inefficiency

6.1 Prototype components (ROS 2 reference)

A minimal prototype consists of: (i) a perception node with three tiers (full/compressed/emergency) exposed behind a common message interface; (ii) a TIC monitor node that computes latency percentiles, miss ratios, uncertainty and drift summaries over a fixed window; (iii) a CAAI supervisor that selects the tier and publishes the active safety envelope mode; and (iv) a safety controller that enforces envelope constraints independently of perception tier.

6.2 Experimental factors and stressors

To exercise worst-case behavior, experiments SHOULD include controlled stressors: CPU contention (background load), GPU contention (concurrent kernels), memory pressure, thermal throttling (sustained load), and network impairment (delay/loss) when off-robot links are present.

6.3 Metrics and reporting

Each critical path SHOULD report: end-to-end latency distribution (p50/p95/p99/max), jitter, deadline miss ratio, consecutive miss bursts, and monitoring overhead (CPU%, memory, and added latency). Perception quality SHOULD be reported with a task metric (e.g., mAP/F1) and with calibration/drift metrics declared in the TIC. Safety behavior SHOULD be reported as envelope mode traces (tier vs. speed/stop cap) with evidence that tier-down occurs before deadlines are violated under stress.

6.4 Baselines

At minimum, compare: (i) fixed Tier 1 (no adaptation), (ii) fixed Tier 2, and (iii) CAAI (adaptive). Where relevant, include a "reactive" baseline that switches tiers only after a deadline miss (to demonstrate the value of predictive margin-based switching).

6.5 Pilot experiment template: induced load and tier switching

A single realistic experiment can be run on one robot compute platform by instrumenting one end-to-end perception path (camera timestamp to detection publication) under induced load. The recommended protocol is: (i) run 5–10 minutes at nominal load; (ii) introduce CPU/GPU contention (background stressors) for the next 5–10 minutes; and (iii) remove the stressors and observe recovery.

Experiment setup (reporting contract).

(i) Platform (CPU/GPU, RAM, OS/kernel, power mode/governors).
(ii) ROS 2 distro, executor, DDS vendor, and QoS for camera/detections.
(iii) The exact path timestamps used (start event, end event, and clock source).
(iv) Deadline D (ms) and how misses are counted.
(v) TIC window definition (duration or sample count) and the exact statistics computed.
(vi) Stressors and pinning.
(vii) Protocol durations (nominal/stress/recovery).
(viii) Number of independent runs N plus confidence intervals across runs.

Conditions. Report three conditions: (A) fixed Tier 1, (B) fixed Tier 2, (C) CAAI enabled. Optionally add (D) a reactive baseline that switches tiers only after a miss.

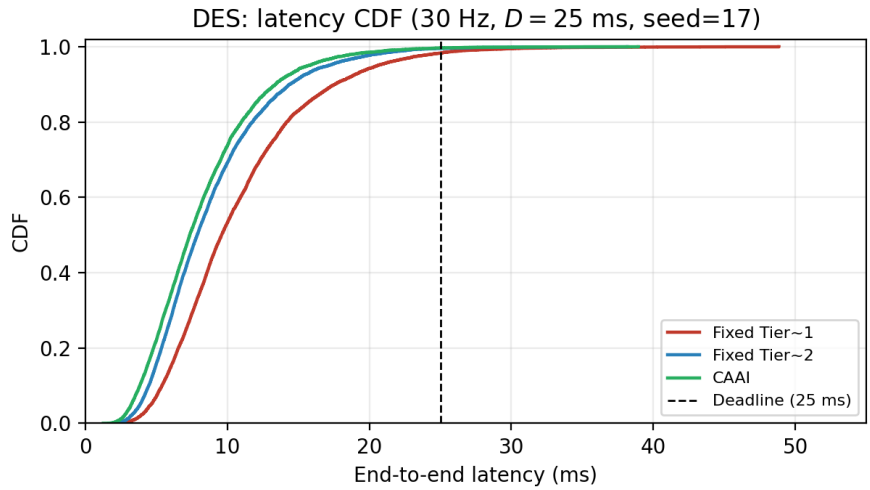

Fig. 8: Discrete-event simulation (Section 6.6): empirical step-CDF of end-to-end latency for fixed Tier 1, fixed Tier 2, and CAAI over the full 300 s run (30 Hz, $D = 25$ ms, seed = 17). The dashed line marks the deadline.

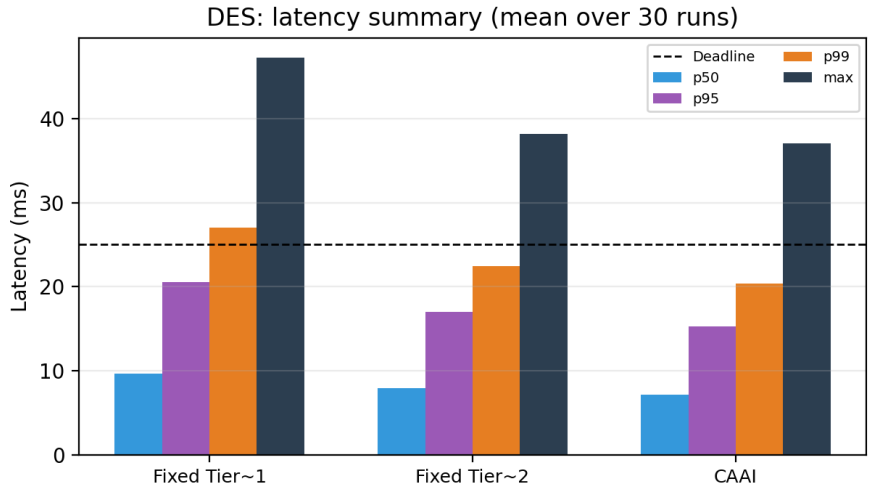

Fig. 9: Discrete-event simulation: mean p50/p95/p99/max latency per condition, averaged over the 30 independent runs reported in Table 11.

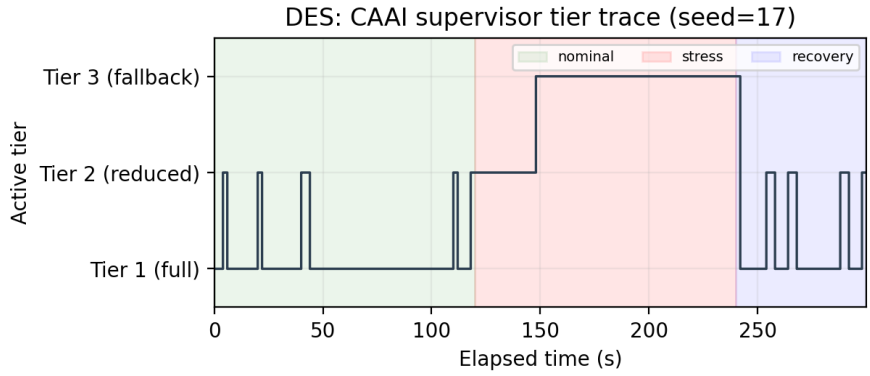

Fig. 10: Discrete-event simulation: CAAI supervisor-selected tier vs. elapsed time for the representative run (seed = 17). Shaded regions mark nominal (0–120 s), stress (120–240 s), and recovery (240–300 s) phases.

6.6 Compact simulation instantiation of TIC/CAAI (reproducible)

To complement the reporting protocol with a concrete instantiation, we implemented a lightweight *discrete-event simulation* of a ROS 2-style publish/subscribe callback chain executed by a single-threaded executor queue. Each cycle produces a message arrival event (30 Hz) that may wait behind previously enqueued callbacks, so end-to-end latency reflects both service time and queueing delay under contention. The three tiers are modeled as independent lognormal service-time draws $X_k \sim \text{LogNormal}(\mu_k, \sigma_k)$ with parameters derived from declared (median, p_{99}) pairs via $\mu_k = \ln(\text{median}_k)$ and $\sigma_k = (\ln p_{99,k} - \mu_k)/z_{0.99}$: Tier 1 $(8.0, 20.0)$ ms $\Rightarrow (\mu_1, \sigma_1) = (2.079, 0.394)$; Tier 2 $(6.5, 15.5)$ ms $\Rightarrow (1.872, 0.374)$; Tier 3 $(3.0, 6.0)$ ms $\Rightarrow (1.099, 0.298)$, chosen to bracket the single-digit-to-$\sim$20 ms typical and tens-of-ms tail callback/chain latencies reported for ROS 2 executor workloads under load [11, 23]. A stress phase applies a 1.35$\times$ service-time multiplier and injects additional blocking time (Gamma-distributed, mean ≈ 4 ms) with probability equal to a phase CPU-utilization parameter (0.05 nominal, 0.45 stress, 0.10 recovery), emulating executor queueing and contention effects consistent with those measurements.

CAAI is implemented as a deterministic supervisor with a rolling-window $p99$ latency margin (margin $= D - p99$), plus uncertainty and drift signals that increase under stress. We report 30 independent simulation runs per condition and compute 95% confidence intervals (bootstrap over runs). A nonparametric Mann–Whitney U test is used for significance on per-run metrics.

The results match the intended behavior of the reference policy: fixed Tier 1 yields the highest tail latency and miss rate under stress, fixed Tier 2 improves timing at a likely quality cost, while CAAI reduces misses by switching away from Tier 1 during stress and returning when margin recovers. In a real

Table 11: Discrete-event simulation results for the camera→detections path (30 Hz, $D = 25$ ms), aggregated across 30 independent runs. Values shown are mean with 95% CI in brackets.

Condition	p99 latency (ms)	deadline miss rate
Fixed Tier 1 (no adaptation)	27.02 [26.90, 27.15]	1.68% [1.64, 1.72]
Fixed Tier 2 (no adaptation)	22.45 [22.33, 22.57]	0.44% [0.42, 0.47]
Reactive after-miss baseline	24.62 [24.52, 24.72]	0.90% [0.87, 0.92]
CAAI (adaptive tiers)	20.39 [20.17, 20.63]	0.24% [0.22, 0.27]

Mann–Whitney U test (per-run): CAAI vs. fixed Tier 1 yields $p = 3.0 \times 10^{-11}$ for $p99$ and $p = 2.9 \times 10^{-11}$ for miss rate.

deployment, the same table would be populated with measured timestamp traces and accompanied by perception-quality and calibration/drift metrics declared in the TIC.

Sensitivity. To test robustness to key switching parameters, we varied the degrade-margin threshold $M_{\text{down12}} \in \{1, 2, 3\}$ ms and the minimum dwell time $\text{dwell}_{\text{min}} \in \{30, 60, 90\}$ cycles. Across these settings, $p99$ remained stable at roughly 20.3–20.5 ms and miss rates remained in the 0.24–0.26% range (see `artifacts/des_outputs/des_sensitivity_caai.csv`), suggesting the supervisor is not brittle to moderate threshold variations in this model.

Threats to validity (simulation). This simulation is intentionally minimal and uses stylized latency/uncertainty/drift processes; therefore, the absolute values in Table 11 should not be interpreted as predictive of any specific robot platform, ROS 2 executor, DDS vendor, or workload. The purpose is to demonstrate how TIC-defined signals can drive a deterministic CAAI policy and produce the reporting artifacts required by Section 6. External validity requires repeating the same analysis with real timestamp traces, declared stressor protocols, and calibrated uncertainty/drift estimators on the target hardware.

6.7 Minimal host-side timing trace under induced CPU load (real measurement)

To complement the simulation with one small real measurement, we executed a portable host-side timing microbenchmark on a development machine (Darwin/arm64, 10 logical CPUs), measuring a 30 Hz periodic "callback" with an end-to-end deadline $D = 25$ ms. The callback performs a fixed compute workload (busy-wait) and records the per-cycle execution latency using a monotonic high-resolution timer. A stress phase adds CPU contention via background burner processes. This microbenchmark is included primarily as an *instrumentation portability demonstration*: the same timestamping and percentile reporting pattern can be applied to ROS 2 executor callbacks and topic traces.

Table 12: Host-side timing trace results (real measurement). Per-phase summaries of per-cycle callback execution latency (ms) at 30 Hz with deadline $D = 25$ ms.

Phase	p50	p95	p99	max	miss rate
Nominal (6 ms work; $n = 900$)	6.00	6.00	6.32	14.00	0.00%
Stress (10 ms work + CPU burners; $n = 900$)	10.00	10.00	10.20	31.98	0.11%
Recovery (6 ms work; $n = 450$)	6.00	6.00	6.52	39.70	0.22%
Pooled ($n = 2250$)	6.00	10.00	10.02	39.70	0.09%

The measurement script used to generate Table 12 is provided as `host_trace_measure.py`. The per-cycle raw trace used for the percentile summaries is also written as a CSV artifact (`artifacts/host_trace_latency_ms.csv`) to support trace-based validation and re-analysis. The same pattern can be adapted to wrap ROS 2 executor callbacks and message timestamps in an actual deployment.

6.8 Reproducibility checklist

A reproducible release SHOULD include: exact hardware/OS/kernel details; ROS 2 version and executor configuration; QoS profiles for each topic; pinned CPU/GPU settings (or explicit statement of defaults); stressor scripts; and raw logs used to compute all reported percentiles and contract-violation events.

7 Cross-Cutting Concerns: Safety, Security, Energy

7.1 Safety Standards and Real-Time Evidence

Robot safety standards provide structured processes for hazard analysis and mandate protective measures, but the safety logic they assume is deterministic. Learning-enabled autonomy breaks that assumption: performance depends on data, operating context, and distribution shift, so assurance cannot stop at design time. Recent analyses call for a shift toward *runtime evidence*, that is, continuous monitoring, logging, and argument maintenance that keeps the safety case aligned with how the system actually behaves in the field [48].

Bridging from probabilistic perception to deterministic envelopes, force limits, separation distances, and approach speeds, usually requires architectural separation. The pattern that has emerged in the literature is monitoring and shielding: a formally specified, deterministic safety monitor intercepts a learned policy's outputs and filters commands that would violate hard constraints [2]. The shield is only as good as its specification, but when specified correctly, it

provides an anchor for certification because it guarantees constraint satisfaction regardless of what the learned component proposes.

TIC captures these ideas explicitly. The safety-fallback clause defines the trigger conditions, degraded modes, and state-machine transitions that the system must implement when monitoring shows that assumptions no longer hold.

7.1.1 Uncertainty-to-safety linkage: a minimal envelope argument

For certification and operational safety, the critical bridge is from probabilistic perception to deterministic constraints (separation distance, braking envelope, speed limits). TIC treats uncertainty and drift metrics as *evidence signals* that gate which tier may issue commands. A compact way to make this auditable is to explicitly connect the uncertainty envelope to a conservative control envelope.

Setup. Let d denote the true distance to the nearest obstacle along the braking direction, and let a perception module produce an estimate $\hat{d}$ plus a scalar uncertainty summary $u \in [0, 1]$ (e.g., a calibrated error bound proxy). Assume the TIC declares a warning threshold U_w and violation threshold U_v such that when $u \leq U_w$ the estimator is in-regime and when $u \geq U_v$ it is out-of-regime. Define a conservative uncertainty margin $\Delta(u)$ that is monotone increasing in u, for example a piecewise-linear map $\Delta(u) = \Delta_w$ for $u \leq U_w$, $\Delta(u) = \Delta_v$ for $u \geq U_v$, with $\Delta_v > \Delta_w$.

Safety invariant (contract form). The controller MUST enforce the deterministic constraint

$$d_{\min}(v) \leq \hat{d} - \Delta(u),$$

where $d_{\min}(v)$ is the required stopping distance at speed v under the applicable braking model. When u grows (poorer evidence), the margin $\Delta(u)$ grows, shrinking the admissible command set. The safety-fallback clause then provides a mechanism when the constraint cannot be maintained (e.g., tier-down to a simpler model or trigger supervised stop).

Role of calibration and drift. If the uncertainty summary is calibrated on the operational distribution, $\Delta(u)$ can be chosen to upper-bound error with high probability; under drift, this assumption degrades. TIC makes that explicit by adding a drift envelope: once drift exceeds its warning/violation thresholds, the system MUST switch to a tier whose safety envelope does not depend on the violated assumption (e.g., slower speed with larger $d_{\min}$, or a conservative geometric detector). This yields an auditable argument shape: (i) define the invariant; (ii) define evidence signals and their bounds; (iii) define monotone margins and tiered envelopes; and (iv) bind violation handling to the fallback state machine.

Fig. 11: A shield enforces constraints such as velocity limits, collision avoidance, on learned commands before actuation. The shield is a deterministic, formally verified component that guarantees safety regardless of policy output. (Author-created figure.)

7.2 Security as a Timing Problem

Cybersecurity in robotics cannot be treated as a purely informational concern: attacks can be translated into physical consequences. In many deployments, an adversary need not take full control of a robot to cause harm; delaying, distorting, or selectively dropping messages can be enough to violate the timing assumptions on which the safety case relies.

The coupling is best seen through a concrete example. If a denial-of-service condition delays a LiDAR scan 50 ms on a 10 Hz pipeline, the effect is not merely "data arrives late"; it is that the planner may act on a world model that is now inconsistent with where obstacles actually are. In a tight human-robot shared workspace, that delay can shrink the effective stopping margin below what the physical envelope requires.

Designing defenses with this coupling in mind pushes teams toward mechanisms that are both security-relevant and timing-aware. Secure boot and signed artifact chains help bound what code can run; network segmentation limits how far an intrusion can propagate; and hardware attestation allows a robot to verify its own integrity at startup and refuse operation when tampering is detected [33]. These measures must be engineered alongside real-time budgets: a defense that adds unpredictable latency to a safety channel can undermine the very guarantees it is meant to protect.

7.3 Energy-Performance Scaling

Energy management is often introduced as a cost topic, but in robotics it becomes a real-time topic because energy-saving mechanisms can change execution timing. Dynamic voltage-frequency scaling reduces power by lowering clock frequency under light load; nonetheless, on hard real-time cores it must be disabled or tightly controlled because frequency transitions and power-management interrupts can introduce jitter that a controller experiences as sampling irregularity.

At the fleet level, the issue broadens. Training and large-scale inference workloads consume substantial energy, and their footprint increasingly matters both operationally and environmentally [31, 43]. A useful architectural observation is that these workloads are not time-coupled to safety-critical control loops: the most expensive compute often runs off-robot, asynchronously, and can be scheduled with fewer deadline constraints than on-device control.

That separation creates leverage. Carbon-aware scheduling can shift batch training or offline evaluation to periods with higher renewable availability, reducing emissions without changing the robot's on-device timing behavior. The discipline remains relevant to real-time design, however, because fleet operators will eventually trade energy against computing headroom; when headroom shrinks, contract-style mechanisms such as TIC and tiered inference become a way to degrade gracefully rather than to miss deadlines unpredictably.

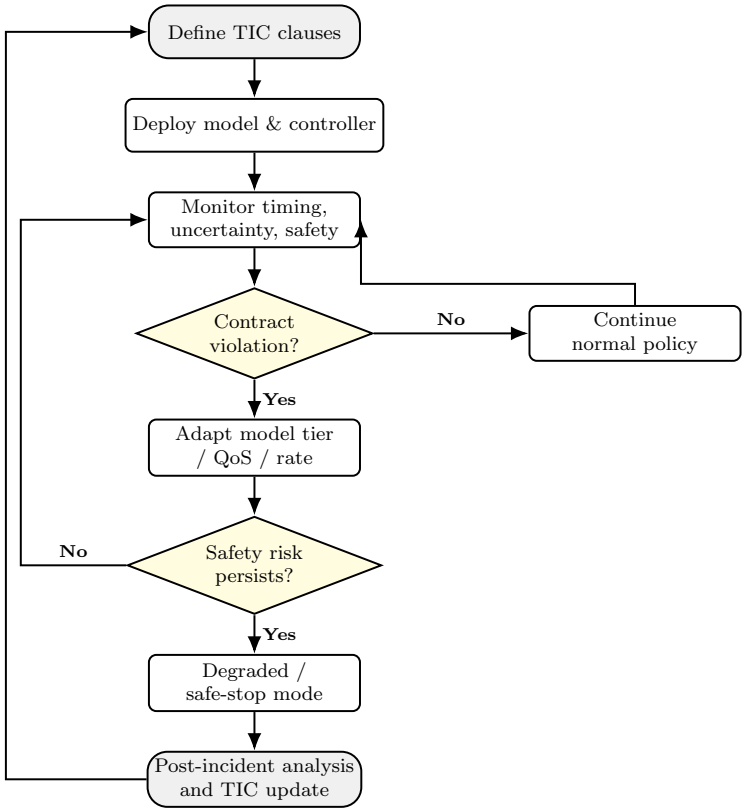

Fig. 12: TIC governance loop: architecture and intelligence are jointly adapted using contract signals, not ad-hoc heuristics. Decision diamonds (yellow) branch on the runtime contract state. The "No violation" path loops back to the monitor immediately. The "Yes" path triggers tier adaptation; if risk persists, a safe stop is engaged, and the TIC is updated before the next deployment cycle.

8 Illustrative Case Patterns

8.1 Autonomous Mobile Robots in Warehouses

Warehouse autonomous mobile robot deployments combine high concurrency with strict human safety requirements and throughput targets. The real-time challenge is maintaining collision-avoidance guarantees while servicing hundreds of concurrent task assignments, where planning delays translate directly into throughput losses. A typical architecture pairs a local navigation stack with hard real-time obstacle avoidance at the edge, and integrates with a warehouse management system layer that handles fleet-level task allocation asynchronously. The key design question is less about optimizing the navigation stack in isolation and more about ensuring that task-assignment failures degrade gracefully without propagating into the navigation layer's timing guarantees.

8.2 Collaborative Robots (Cobots)

In collaborative robot deployments where humans share workspace with robot arms, the dominant real-time concern is minimizing contact and pinch hazards under dynamic human motion. Fast depth-camera and skeleton-estimation pipelines operating at 30 Hz or above, feeding into certified stopping behaviors with documented reaction times, form the core safety mechanism. Real-time intelligence in this context focuses on monitoring and reactive stopping rather than maximizing task throughput, because a single unsafe contact event is a more consequential failure than a throughput reduction.

8.3 Field Robots

Field robots operating in outdoor or unstructured environments face a connectivity profile that is fundamentally different from warehouse deployments: intermittent and unpredictable network availability means that cloud-dependent functions must be confined to asynchronous batch operations, while all safety-critical and navigation functions must operate fully autonomously at the edge. Map and model weight updates are deferred to scheduled charging intervals, eliminating the need for continuous cloud connectivity.

9 Original Contributions

Relative to the timing monitors, middleware practices, and learning-deployment patterns surveyed in Section 2, this paper adds three artifacts that are specified jointly rather than as disconnected guidelines:

– **TIC** (Sections 4.6–4.6.2): a versioned, machine-readable contract co-specifying latency, uncertainty/drift, fallback, observability, and scalability obligations with required runtime evidence.

Table 13: Case pattern summary.

Domain	Dominant real-time concern	Scalability lever
Warehouse AMR	Human safety + traffic	Fleet manager + rules
Cobots	Contact / pinch hazards	Fast sensing + certified stops
Drones (civil)	Latency + geofencing	Offboard airspace services
Industrial arms	Cycle time + repeatability	Deterministic bus + PLC

– **CAAI** (Section 5.4): a deterministic tier supervisor whose downgrade/recovery rules are driven by TIC monitor signals and coupled to safety envelopes.
– **Adoption package** (Section 6): a reporting protocol, discrete-event instantiation with statistical analysis (Section 6.6), and a reference monitor/schema for independent implementation.

The survey synthesis and case patterns (Sections 2–8) contextualize these artifacts; they are not separate empirical claims.

10 Challenges and Future Directions

Despite the architectural and algorithmic progress surveyed here, several fundamental challenges remain open. The discussion below groups these by theme, focusing on where the most productive near-term research lies rather than attempting an exhaustive enumeration.

10.1 Limitations and Scope

This paper's contribution is a contract specification and reference architecture, not an empirical benchmark. The DES in Section 6.6 demonstrates the mechanics of TIC-governed tier switching under a validated queueing model; hardware validation of specific platforms is intentionally left to adopters using the reporting protocol in Section 6. Parameters such as TIC clause thresholds, RTI-Score weights, and CAAI switching constants are design-time declarations that must be derived from system-level hazard analysis and calibrated on the target platform. Drift and uncertainty signals are proxies whose operational reliability depends on the chosen estimators and the quality of instrumentation. The paper does not claim to solve worst-case execution-time bounding for modern deep networks; instead it supplies architectural patterns and runtime monitoring hooks that make timing risk explicit and manageable in practice.

10.2 Verification, Safety, and Formal Guarantees

The central unsolved problem in deploying learned components in safety-critical loops is formal verification at scale. SMT-based neural network verification approaches [38] scale only to small, shallow networks; abstract interpretation remains over-approximate for nonlinear activation functions, and the verification runtimes for transformer-scale models are orders of magnitude beyond what is practical. A productive intermediate direction is assume-guarantee decomposition: formally verify the deterministic control layers, treat the learned component as a black box with bounded output deviation, and propagate that deviation bound through TIC latency and uncertainty envelopes to determine whether the overall system's safety case holds [9]. This does not resolve verification for the learned component itself, but it allows a meaningful safety argument to be constructed before verification tools catch up with the scale of contemporary models.

The certifiability of end-to-end learned stacks presents a related and equally pressing challenge. Systems that collapse multiple classical pipeline stages into a single unified policy create qualitatively new difficulties for regression testing, failure mode analysis, and evidence production for regulatory approval [9, 38]. Existing safety standards were not written with such architectures in mind, and the process of extending or reinterpreting them for learning-enabled systems is ongoing in multiple standards bodies simultaneously. Contract synthesis automation, generating TIC clauses directly from mission goal specifications and hazard analyses rather than requiring domain engineers to derive them by hand, would help close the gap between safety engineering practice and software deployment speed.

10.3 Benchmarking, Sensing, and Sustainability

A measurement gap in existing literature limits the practical utility of published latency results: reported figures are almost universally average-case measurements on specific hardware under light load. No community-accepted benchmark currently reports worst-case execution time, 99th-percentile latency under realistic competing load, or latency degradation under input-distribution shift, across a representative range of heterogeneous edge SoC platforms [3]. System designers would benefit directly from such benchmarks, provided they are built on standardized workloads and measurement protocols.

Event cameras, sensors that report per-pixel intensity changes asynchronously at microsecond resolution rather than outputting full frames at a fixed rate, represent a promising direction for reducing the effective latency of the perception layer [17]. On static scenes their data rate falls by two to three orders of magnitude compared to frame cameras, while during high-speed motion they retain full temporal resolution. Integrating the uncertainty characteristics of event-based perception into TIC uncertainty envelopes is an open systems design challenge requiring new calibration methodologies.

Energy consumption in large-scale model training and fleet-level inference is emerging as a first-order engineering constraint, not just a cost consideration. Carbon-aware job scheduling strategies that shift training workloads to grid windows of high renewable availability have been shown to substantially reduce training-related emissions without degrading policy quality [13, 31, 43]. Extending these strategies to cover the full energy lifecycle of a robotic deployment, from component manufacturing to operational inference, requires improved tooling and finer-grained energy accounting at each tier of the edge-cloud continuum.

10.4 Human Factors and Cross-Fleet Transfer

Mixed-initiative systems in which a human operator and a robot share control or oversight responsibility introduce a requirement that the TIC framework has not yet fully addressed: the robot must communicate its current operating tier, confidence level, and time-to-next-safe-stop threshold in a form interpretable to a non-specialist operator within that operator's own reaction time. Addressing this requires input from cognitive ergonomics research that has yet to be fully integrated into the real-time robotics literature. Endsley's three-level situation awareness model provides measurable criteria for evaluating whether an operator can accurately interpret the robot's current operating tier within their own reaction-time budget [14]. Applying these criteria to tier-status display design is a tractable near-term direction, as is adapting Vicente and Rasmussen's ecological interface design framework to represent the TIC state in ways that align with operators' mental models [50].

Cross-fleet policy transfer, deploying a policy trained and validated in one operational environment to a fleet in a different environment, requires characterizing the gap between source and target operational design domains, bounding the performance degradation this gap introduces, and propagating updated bounds through the TIC clauses of all affected components before the policy is activated [9]. Doing this without incurring unsafe negative transfer, where the transferred policy performs worse than the policy it replaces, remains an open question with significant practical consequences for multi-site deployments.

11 Conclusion

At the deepest level, real-time intelligence and scalable architecture both respond to the same irreducible constraint: the physical world operates on its own clock, and it does not pause while a system computes. The arguments developed here lead to a view in which timing discipline and architectural scalability are not competing objectives to be traded off against each other, but complementary requirements that must be addressed jointly from the earliest stages of system design.

The three-layer control hierarchy (Figure 5), the timing-tier taxonomy (Table 10), the Temporal Intelligence Contract, and the Contract-Aware Adaptive

Inference mechanism together provide a principled framework. Intelligence without timing discipline risks unsafe behavior that no accuracy improvement can correct after the fact; architecture without scalability strands promising algorithms in prototype environments where they cannot accumulate the operational evidence needed to justify production deployment. Practitioners should evaluate systems against explicit latency budgets, with structural separation of safety-critical loops from higher-level intelligence, with middleware QoS choices matched to the semantic requirements of each channel, and with operational monitoring models that connect edge autonomy to fleet-level learning in ways that preserve safety invariants across the full deployment lifecycle.

The next decade will likely see verified real-time AI emerge as a practical engineering discipline rather than a theoretical aspiration, driven by the convergence of improved formal tools, better standardized benchmarks for worst-case inference latency, and regulatory pressure that creates commercial incentives for rigorous safety evidence. Tighter hardware-software co-design, in which model architecture and silicon microarchitecture evolve together, will push the achievable frontier of real-time perception quality. The TIC perspective offers a concrete bridge from theory to deployment by making timing, uncertainty, safety, and scalability obligations explicit, machine-readable, and continuously monitored, embedding the engineering discipline that safety requires directly into the operational fabric of the system.

References

1. Ali A, Tawfik MM, Saafan MM (2026) Cross-dataset late fusion of camera–LiDAR and radar models for object detection. Scientific Reports 16:879, DOI 10.1038/s41598-025-32588-5
2. Alshiekh M, Bloem R, Ehlers R, Könighofer B, Niekum S, Topcu U (2018) Safe reinforcement learning via shielding. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)
3. Amalou AN, Fromont E, Puaut I (2023) CAWET: Context-aware worst-case execution time estimation using transformers. In: 35th Euromicro Conference on Real-Time Systems (ECRTS 2023), LIPIcs, DOI 10.4230/LIPIcs.ECRTS.2023.7
4. Årzén KE (1999) A simple event-based pid controller. In: Proceedings of the 14th IFAC World Congress, Beijing, China
5. AUTOSAR Consortium (2020) Specification of Timing Extensions. AUTOSAR, release r20-11 edn, URL `https://www.autosar.org/fileadmin/standards/R20-11/CP/AUTOSAR_TPS_TimingExtensions.pdf`
6. Bernat G, Burns A, Llamosí A (2001) Weakly hard real-time systems. IEEE Transactions on Computers 50(4):308–321, DOI 10.1109/12.919277
7. Bifet A, Gavaldà R (2007) Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining (SDM), SIAM, pp 443–448

8. Brooks RA (1986) A robust layered control system for a mobile robot. IEEE Journal of Robotics and Automation 2(1):14–23, DOI 10.1109/JRA.1986.1087032
9. Brunke L, Greeff M, Hall AW, Yuan Z, Zhou S, Panerati J, Schoellig AP (2022) Safe learning in robotics: From learning-based control to safe reinforcement learning. Annual Review of Control, Robotics, and Autonomous Systems 5:411–444, DOI 10.1146/annurev-control-042920-020211
10. Buttazzo GC (2011) Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications. Springer
11. Casini D, Blaß T, Lütkebohle I, Brandenburg BB (2019) Response-time analysis of ros 2 processing chains under reservation-based scheduling. In: 31st Euromicro Conference on Real-Time Systems (ECRTS 2019), LIPIcs, DOI 10.4230/LIPIcs.ECRTS.2019.6
12. Dean T, Boddy M (1988) An analysis of time-dependent planning. In: Proceedings of the 7th National Conference on Artificial Intelligence (AAAI), pp 49–54
13. Dodge J, Prewitt T, Tachet des Combes R, Sloggett E, Strubell E, Smith NA, Schwartz R (2022) Measuring the carbon intensity of AI in cloud instances. In: Proc. ACM FAccT, pp 1877–1894, DOI 10.1145/3531146.3533234
14. Endsley MR (1995) Toward a theory of situation awareness in dynamic systems. Human Factors 37(1):32–64, DOI 10.1518/001872095779049543
15. Ferrando A, Cardozo N, Serventi A, Brugali D (2020) ROSMonitoring: A runtime verification framework for ROS. In: Towards Autonomous Robotic Systems (TAROS), Springer, Lecture Notes in Computer Science, DOI 10.1007/978-3-030-63486-6_7
16. Gal Y, Ghahramani Z (2016) Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: Proceedings of the 33rd International Conference on Machine Learning, PMLR, vol 48, pp 1050–1059, DOI 10.5555/3045390.3045502
17. Gallego G, Delbrück T, Orchard G, Bartolozzi C, Taba B, Censi A, Leutenegger S, Davison AJ, Conradt J, Daniilidis K, Scaramuzza D (2022) Event-based vision: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence 44(1):154–180, DOI 10.1109/TPAMI.2020.3008413
18. Gerkey BP, Matarić MJ (2004) A formal analysis and taxonomy of task allocation in multi-robot systems. The International Journal of Robotics Research 23(9):939–954, DOI 10.1177/0278364904045564
19. Gretton A, Borgwardt KM, Rasch MJ, Schölkopf B, Smola A (2012) A kernel two-sample test. Journal of Machine Learning Research 13:723–773
20. Guo C, Pleiss G, Sun Y, Weinberger KQ (2017) On calibration of modern neural networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR, vol 70, pp 1321–1330
21. Huang J, Erdogan C, Zhang Y, Moore B, Luo Q, Sundaresan A, Rosu G (2014) ROSRV: Runtime verification for robots. In: Runtime Verification (RV 2014), Springer, Lecture Notes in Computer Science, vol 8734, pp 247–254, DOI 10.1007/978-3-319-11164-3_20

22. Huang P, Zeng L, Chen X, Luo K, Zhou Z, Yu S (2022) Edge robotics: Edge-computing-accelerated multirobot simultaneous localization and mapping. IEEE Internet of Things Journal 9(15):14087–14102, DOI 10.1109/JIOT.2022.3146461, earlier version: arXiv:2112.13222, 2112.13222
23. Kronauer T, Pohlmann J, Matthé M, Smejkal T, Fettweis G (2021) Latency analysis of ROS2 multi-node systems. In: Proc. IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI), Karlsruhe, Germany, pp 1–7, DOI 10.1109/MFI52462.2021.9591166
24. Kuffner JJ (2010) Cloud-enabled robots. In: Proceedings of the 10th IEEE-RAS International Conference on Humanoid Robots (Humanoids)
25. Liu JWS (2000) Real-Time Systems. Prentice Hall, Upper Saddle River, NJ
26. Maruyama Y, Kato S, Azumi T (2016) Exploring the performance of ros2. In: Proceedings of the International Conference on Embedded Software (EMSOFT)
27. Miskowicz M (ed) (2015) Event-Based Control and Signal Processing. CRC Press, DOI 10.1201/b18550
28. Niu Y, Chen B, Li R (2018) Optimal sensor scheduling for state estimation with communication energy constraint. IEEE Transactions on Automatic Control 63(9):3100–3107, DOI 10.1109/TAC.2017.2787545
29. Open Robotics (2024) Executors. Online, URL https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Executors.html
30. Palau CE, et al. (2024) Safety-critical edge robotics architecture for low-latency and deterministic operation. arXiv preprint arXiv:2406.14391, 2406.14391
31. Patterson D, Gonzalez J, Le Q, Liang C, Munguiá LM, Rothchild D, So D, Texier M, Dean J (2021) Carbon emissions and large neural network training. arXiv preprint arXiv:2104.10350, 2104.10350
32. Pfeiffer B, Suppa M (2017) 5G for robotics: Ultra-low latency control of distributed robotic systems. In: Proc. International Symposium on Systems Engineering (ISSE), Vienna, Austria, DOI 10.1109/SYSOSE.2017.8088274
33. Pu CQ, Wang K, Xu Q, Wu J, Yu Y, Zhao X, Liu F (2023) Cybersecurity for robotics: A comprehensive review. Array 18:200237, DOI 10.1016/j.array.2023.200237
34. Quigley M, Conley K, Gerkey B, Faust J, Foote T, Leibs J, Wheeler R, Ng AY (2009) Ros: an open-source robot operating system. In: ICRA Workshop on Open Source Software
35. Restuccia F, Biondi A (2021) Time-predictable acceleration of deep neural networks on fpga soc platforms. In: 2021 IEEE Real-Time Systems Symposium (RTSS), pp 441–454, DOI 10.1109/RTSS52674.2021.00047
36. ROS 2 Real-time Working Group (2023) Real-time programming in ROS 2. Online, URL https://ros-realtime.github.io/RAP/
37. Ross S, Gordon G, Bagnell D (2011) A reduction of imitation learning and structured prediction to no-regret online learning. In: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)

38. Seshia SA, Sadigh D, Sastry SS (2022) Toward verified artificial intelligence. Communications of the ACM 65(7):46–55, DOI 10.1145/3486641
39. Shah S, Dey D, Lovett C, Kapoor A (2018) Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In: Field and Service Robotics: Results of the 11th International Conference, Springer Proceedings in Advanced Robotics, pp 621–635, DOI 10.1007/978-3-319-67361-5_40
40. Shi W, Cao J, Zhang Q, Li Y, Xu L (2016) Edge computing: Vision and challenges. IEEE Internet of Things Journal 3(5):637–646, DOI 10.1109/JIOT.2016.2579198
41. Shih CS, Liu JWS (1995) Algorithms for scheduling imprecise computations with timing constraints to minimize maximum error. IEEE Transactions on Computers 44(3):466–471
42. Siddiqi N (2005) Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Wiley
43. Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the ACL, pp 3645–3650, DOI 10.18653/v1/P19-1355
44. Sun P, Kretzschmar H, Dotiwalla X, et al. (2020) Scalability in perception for autonomous driving: Waymo open dataset. In: Proc. IEEE/CVF CVPR, DOI 10.1109/CVPR42600.2020.00252
45. Sze V, Chen YH, Yang TJ, Emer JS (2017) Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE 105(12):2295–2329, DOI 10.1109/JPROC.2017.2761740
46. Thrun S, Burgard W, Fox D (2005) Probabilistic Robotics. MIT Press
47. Tobin J, Fong R, Ray A, Schneider J, Zaremba W, Abbeel P (2017) Domain randomization for transferring deep neural networks from simulation to the real world. In: Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, pp 23–30, DOI 10.1109/IROS.2017.8202133
48. Underwriters Laboratories (2023) UL 4600: Standard for safety for the evaluation of autonomous products. 2nd ed., UL Standards & Engagement
49. Urbaniak D, et al. (2024) Distributed control for collaborative robotic systems using 5G edge computing. IEEE Access DOI 10.1109/ACCESS.2024.3475584
50. Vicente KJ, Rasmussen J (1992) Ecological interface design: Theoretical foundations. IEEE Transactions on Systems, Man, and Cybernetics 22(4):589–606, DOI 10.1109/21.156574
51. Viktor P, Kiss G (2026) Sensors in self-driving vehicles: A detailed literature review and new trends. Sensors 26(7):2153, DOI 10.3390/s26072153
52. Wurman PR, D'Andrea R, Mountz M (2008) Coordinating hundreds of cooperative, autonomous vehicles in warehouses. AI Magazine 29(1):9–20, DOI 10.1609/aimag.v29i1.2082
53. Yan D, Li R, Xiong W, Huang X (2026) Sensor selection strategy and multimodal layout optimization for autonomous vehicles: A review. Measurement 274:121225, DOI 10.1016/j.measurement.2026.121225, in press

54. Yu X, Peña Queralta J, Heikkonen J, Westerlund T (2021) Federated learning in robotic and autonomous systems. Procedia Computer Science 191:135–142, DOI 10.1016/j.procs.2021.07.041, URL https://arxiv.org/abs/2104.10141, 2104.10141
55. Zhang T, Wang G, Xue C, Wang J, Nixon M, Han S (2023) Time-sensitive networking (TSN) for industrial automation: Current advances and future directions. ACM Computing Surveys 57(2):1–38, DOI 10.1145/3695248
56. Zhao W, Queralta JP, Westerlund T (2020) Sim-to-real transfer in deep reinforcement learning for robotics: A survey. In: Proc. IEEE Symposium Series on Computational Intelligence (SSCI), Canberra, Australia, pp 737–744, DOI 10.1109/SSCI47803.2020.9308468
57. Zilberstein S (1996) Using anytime algorithms in intelligent systems. AI Magazine 17(3):73–83